Second Normal Form (2NF) in DBMS

Complete Theory, Partial Dependency, Decompositions, Short-cuts & GATE Practice

Second Normal Form (2NF) is built directly upon core relational concepts: Functional Dependency, Candidate Keys, Prime/Non-Prime Attributes, and Attribute Closures. Its primary goal is to eliminate redundant data caused by partial dependencies in composite keys.

1. What is Second Normal Form?

A relation R is in Second Normal Form (2NF) if it satisfies two mandatory conditions:

Rule 1: The relation must already be in **First Normal Form (1NF)** (all attributes contain atomic values).

Rule 2 ⭐: There must be **NO Partial Dependency** of any non-prime attribute on any candidate key.

Formal Definition: Every non-prime attribute must be *fully functionally dependent* on every candidate key of the relation.

2. Understanding Partial Dependency

Partial dependency is the core concept tested in 2NF questions.

Suppose a relation has a composite candidate key AB. Its proper subsets are $\{A\}$ and $\{B\}$.

If there exists a functional dependency $A \rightarrow C$ where C is a non-prime attribute, then C depends on only *part* of the candidate key AB. This is called a **Partial Dependency**.

Partial Dependency Pattern ⭐:
Candidate Key = AB
Dependency: $A \rightarrow C$ (where C is non-prime)
$\Rightarrow C$ is determined by a proper subset (A) of the candidate key AB.

3. Full vs. Partial Functional Dependency

Let AB be the candidate key and C be a non-prime attribute:

- **Full Functional Dependency:** $AB \rightarrow C$ holds true, and *neither* $A \rightarrow C$ nor $B \rightarrow C$ holds true. That is, the complete candidate key is necessary to determine C.
- **Partial Functional Dependency:** $AB \rightarrow C$ holds true, BUT $A \rightarrow C$ (or $B \rightarrow C$) also holds true. Here, C can be determined by a part of the candidate key alone.

4. The Golden Identification Test for 2NF Violation ⭐⭐⭐

To identify if an FD $X \rightarrow Y$ causes a partial dependency violation, verify both conditions:

$$LHS \subset Candidate\ Key \quad \& \quad RHS = Non\text{-}Prime\ Attribute$$

If **even one** functional dependency satisfies both conditions above, the relation **FAILS 2NF**.

5. Real-World Example: Customer & Store

Consider the unnormalized table below:

Customer_ID	Store_ID	Location
C1	S1	Delhi
C2	S1	Delhi
C3	S2	Bangalore
C4	S3	Mumbai

- **Candidate Key:** $\{Customer\_ID, Store\_ID\}$
- **Prime Attributes:** $Customer\_ID, Store\_ID$
- **Non-Prime Attribute:** $Location$

Notice that $Store\_ID \rightarrow Location$ holds true (e.g., $S1$ is always Delhi). Since $Store\_ID$ is a proper subset of the composite candidate key $\{Customer\_ID, Store\_ID\}$, $Store\_ID \rightarrow Location$ is a **Partial Dependency**. Therefore, this table is **NOT in 2NF**.

6. How to Convert to 2NF? (Decomposition)

To eliminate partial dependencies, decompose the table into separate relations where every non-prime attribute depends on a full candidate key:

Table 1: Customer_Store (2NF)

Customer_ID	Store_ID
C1	S1
C2	S1
C3	S2
C4	S3

Primary Key: $\{Customer\_ID, Store\_ID\}$

Table 2: Store (2NF)

Store_ID	Location
S1	Delhi
S2	Bangalore

Store_ID	Location
S3	Mumbai

Primary Key: $Store\_ID$

Benefits of Decomposition:
- **Less Redundancy:** Store locations are stored only once per store rather than repeated for every customer.
- **Anomaly Elimination:** Updates to a store's location only happen in one place.
- **No Partial Dependency:** Both tables now satisfy 2NF.

7. Step-by-Step GATE-Style Problem

Given relation $R(A, B, C, D, E, F)$ with functional dependencies:

$$C \rightarrow F, \quad E \rightarrow A, \quad EC \rightarrow D, \quad A \rightarrow B$$

Step 1: Find the Candidate Key(s)

Look at attributes missing from the RHS of all FDs: Attributes E and C never appear on the RHS. Therefore, EC MUST be part of every candidate key.

Compute closure of EC:

- $(EC)^+ = \{E, C\}$
- From $C \rightarrow F \Rightarrow$ add $F \Rightarrow \{E, C, F\}$
- From $E \rightarrow A \Rightarrow$ add $A \Rightarrow \{E, C, F, A\}$
- From $EC \rightarrow D \Rightarrow$ add $D \Rightarrow \{E, C, F, A, D\}$
- From $A \rightarrow B \Rightarrow$ add $B \Rightarrow \{E, C, F, A, D, B\} = R$

Since $(EC)^+ = R$, EC is the **Candidate Key**.

Step 2: Classify Attributes

- **Prime Attributes** (Part of candidate key): $\{E, C\}$
- **Non-Prime Attributes** (Not in candidate key): $\{A, B, D, F\}$

Step 3: Test Each Functional Dependency for Partial Dependency

FD	LHS Condition ($LHS \subset CK$?)	RHS Condition ($RHS = Non\text{-}Prime$?)	Partial Dependency?
$C \rightarrow F$	YES ($C \subset EC$)	YES (F is non-prime)	PARTIAL DEPENDENCY ❌
$E \rightarrow A$	YES ($E \subset EC$)	YES (A is non-prime)	PARTIAL DEPENDENCY ❌
$EC \rightarrow D$	NO (EC is full CK)	YES (D is non-prime)	Full Dependency (OK)
$A \rightarrow B$	NO ($A \notin EC$)	YES (B is non-prime)	Not Partial re: CK (OK)

Conclusion: Since $C \rightarrow F$ and $E \rightarrow A$ are partial dependencies, relation R is **NOT in 2NF**.

8. Extremely Important Exam Shortcut ⭐

Single-Attribute Candidate Key Rule:
If all candidate keys of a relation consist of a **single attribute** (e.g., CK = A), partial dependency is **IMPOSSIBLE**.

Why? The only proper subset of $\{A\}$ is $\emptyset$ (empty set). No functional dependency can have an empty LHS.

⚡ **Shortcut:** Single-Attribute Key + 1NF ⇒ **AUTOMATICALLY IN 2NF!**

9. Complete 2NF Verification Workflow

Start Relation R
↓
Is it in 1NF? ⇒ *No* ⇒ NOT 2NF
↓ ***Yes***
Find Candidate Keys (CKs)
↓
Identify Prime & Non-Prime Attributes
↓
Check each FD: Is $LHS \subset CK$ AND $RHS = Non\text{-}Prime$?
↓
If YES for ANY FD ⇒ NOT in 2NF | If NO for ALL FDs ⇒ IS IN 2NF

10. Master Summary Cheat Sheet

Concept	Core Rule / Rule Formula
Golden 2NF Formula	$ext\{1NF\} + ext\{No\ Partial\ Dependency\} = ext\{2NF\}$
Partial Dependency Definition	$LHS \subset ext\{Candidate\ Key\} \quad ext\{AND\} \quad RHS = ext\{Non\text{-}Prime\ Attribute\}$
Prime Attribute	An attribute that is part of *at least one* candidate key
Non-Prime Attribute	An attribute that is *not part of any* candidate key
Single Attribute CK	Partial dependency is impossible ⇒ Automatically in 2NF
2NF Solution	Decompose relation into smaller relations around partial FDs